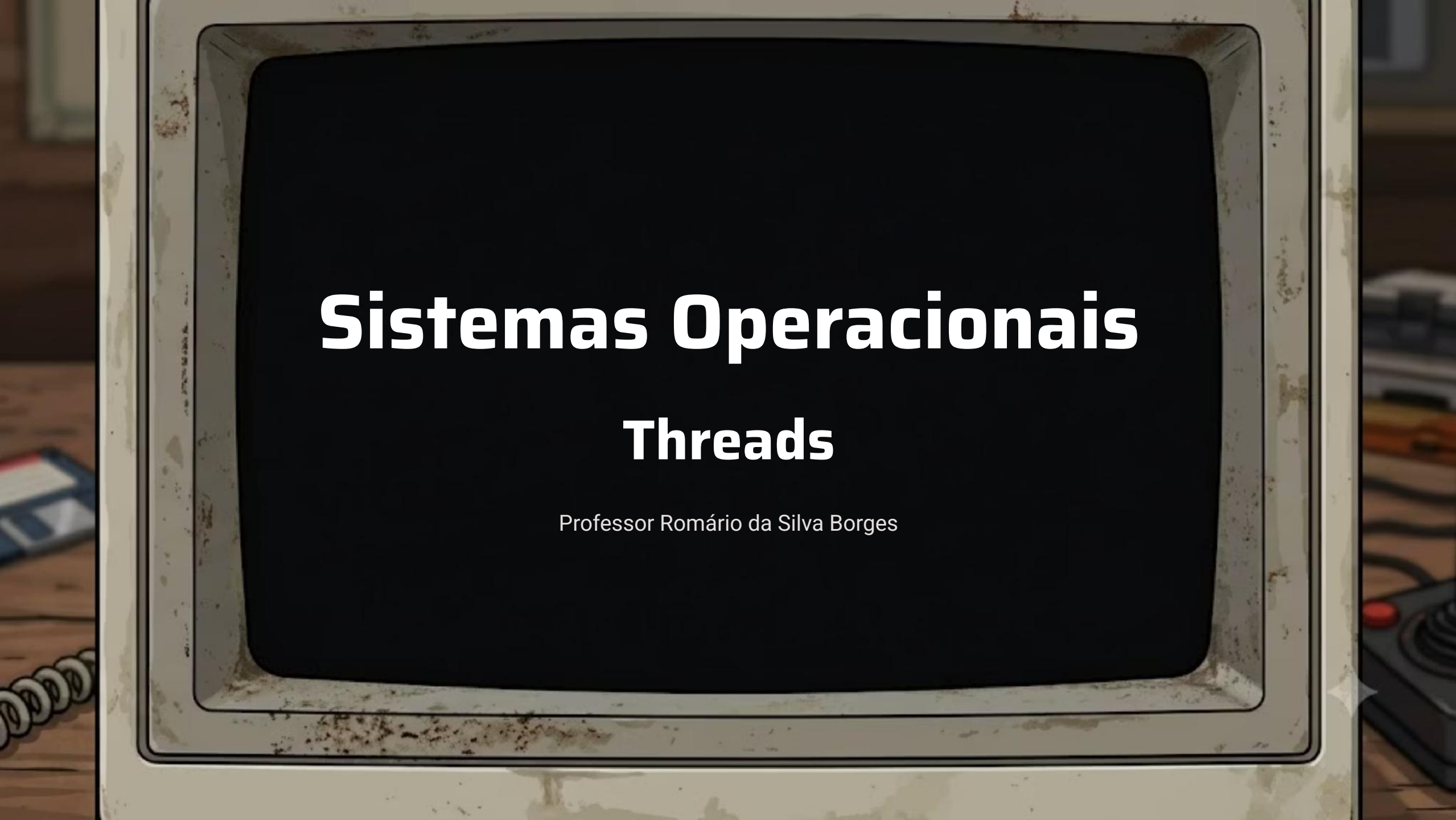




Sistemas Operacionais

Threads

Professor Romário da Silva Borges



Se um processo já pode executar um programa inteiro, por que sistemas modernos precisam dividir esse mesmo processo em várias linhas de execução?

Threads

Recapitulando...

Processo

Definição

Programa em execução.

Espaço próprio

Possui **espaço de endereçamento** isolado.

Agrupa recursos

Arquivos abertos, variáveis, memória.

Controlado pelo SO

O sistema operacional gerencia seu ciclo de vida.

Isolamento é bom para:

- Segurança
- Consistência
- Proteção da memória

Mas pode dificultar:

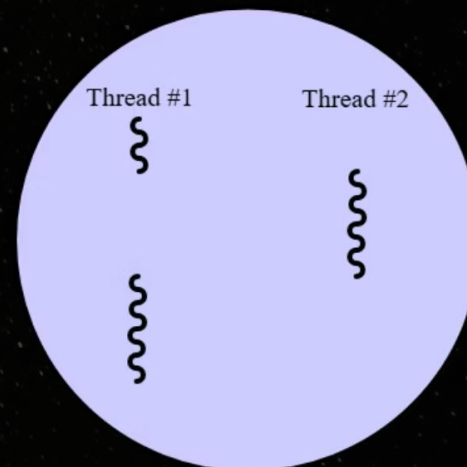
- Compartilhamento de dados
- Atividades simultâneas na mesma aplicação
- Criação rápida de unidades de execução

Thread

Uma **thread** é uma **linha de execução** dentro de um processo. Ela compartilha recursos do processo, mas possui seu próprio fluxo de execução.

⚡ **Unidade básica de uso da CPU**

🪶 **Leves e bom desempenho**



Analogia do que seria um processo e várias threads relacionadas:



Recursos da thread

Contador de programa

Aponta para a próxima instrução a executar.

Pilha de execução

Armazena variáveis locais e chamadas de função.

Registradores

Guardam o estado atual do processador para a thread.

Estado e escalonamento

Permite que a thread seja interrompida e retomada.

Recursos compartilhados



Código

Todas as threads executam o mesmo código do processo.



Dados globais

Variáveis globais e heap acessíveis por todas.



Arquivos abertos

Descritores de arquivo e sinais do processo.



compartilhamento exige **sincronização**.

Threads em nível de usuário

São implementadas no espaço do usuário por meio de bibliotecas específicas. O sistema operacional não tem conhecimento direto dessas threads — ele enxerga apenas o processo como um todo.

Threads em nível de núcleo

São gerenciadas diretamente pelo sistema operacional. Nesse caso, cada thread é visível ao núcleo, que pode gerenciá-las.

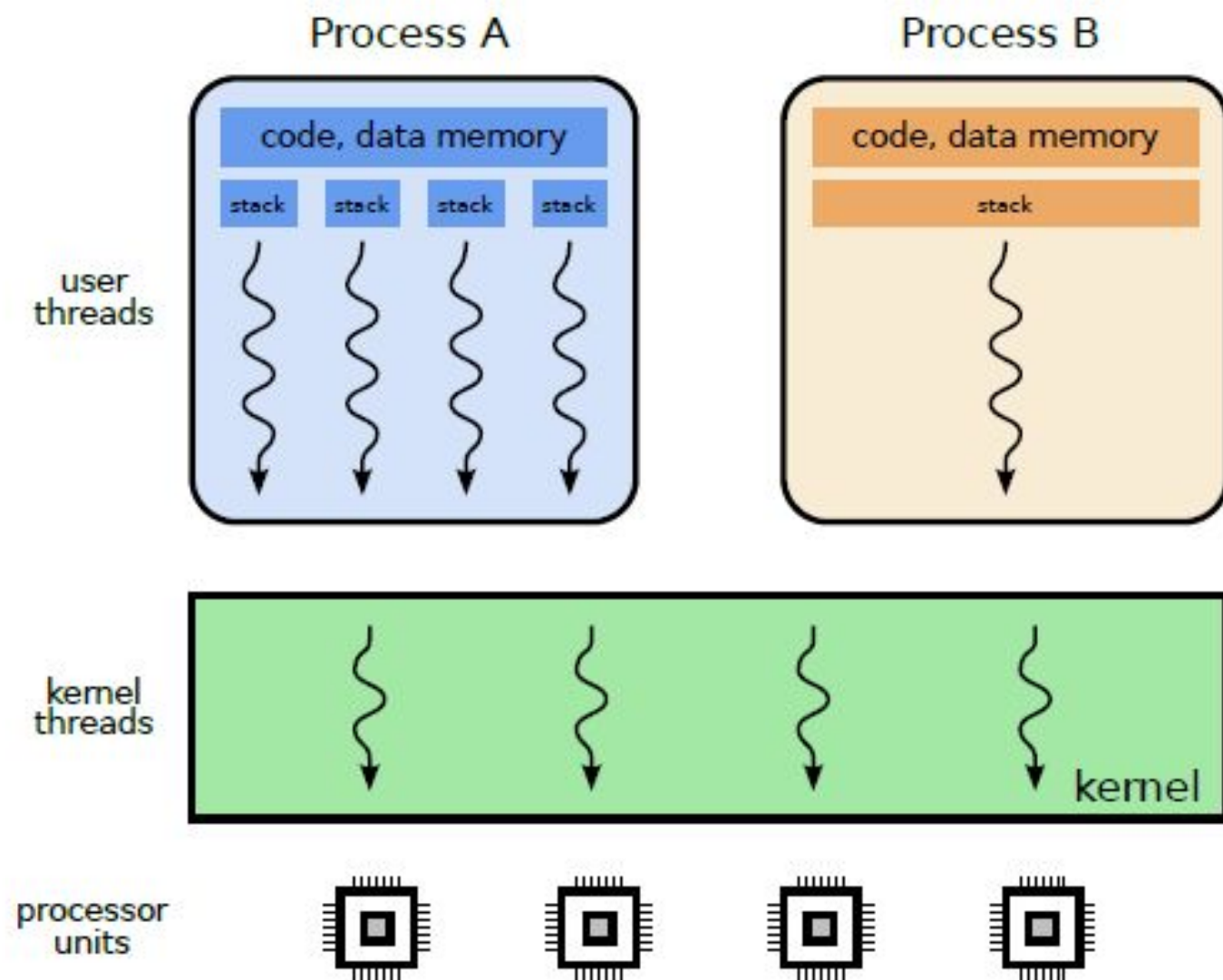
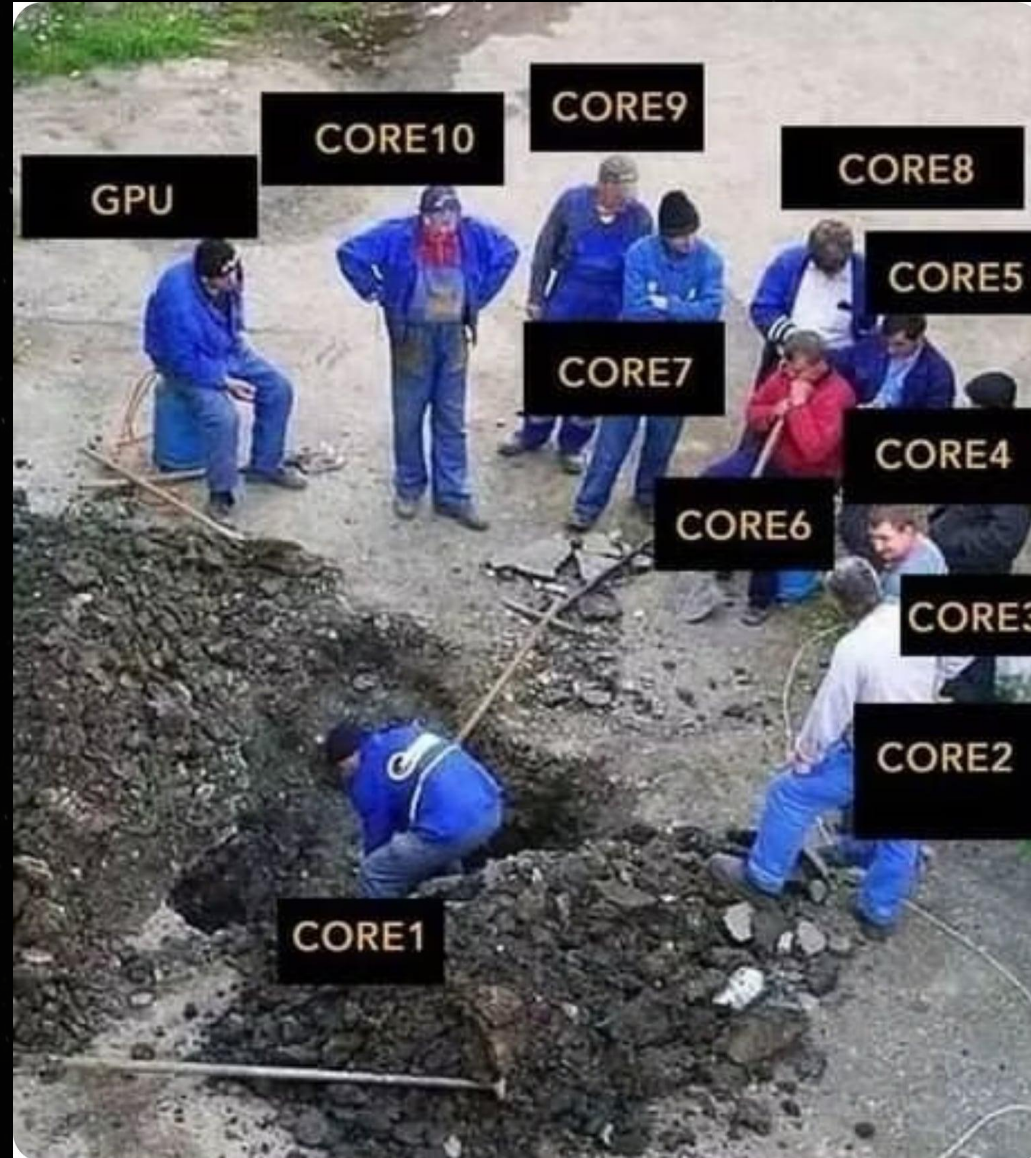


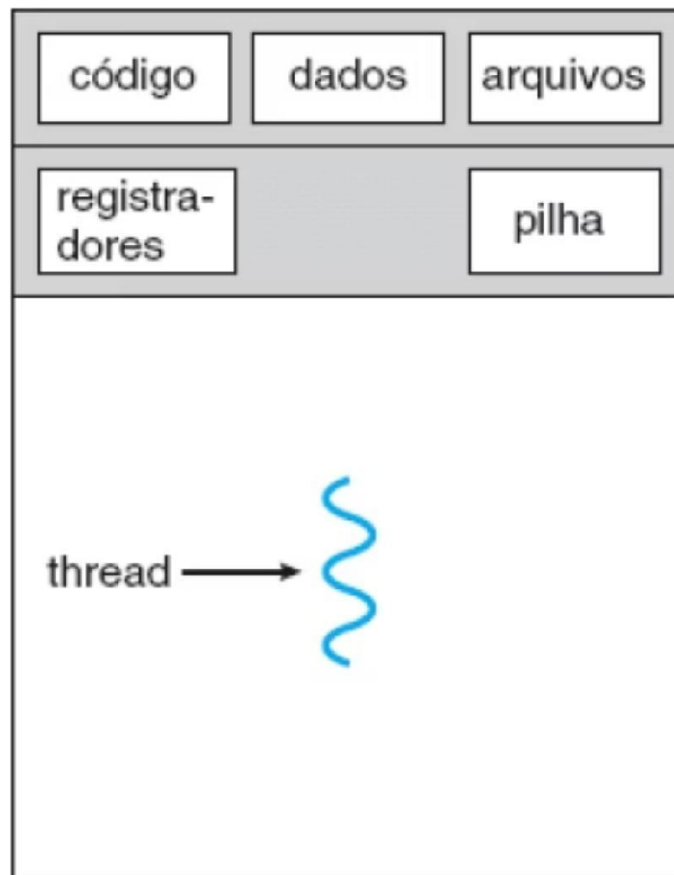
Figura 5.4: *Threads* de usuário e de núcleo.

Multithreading

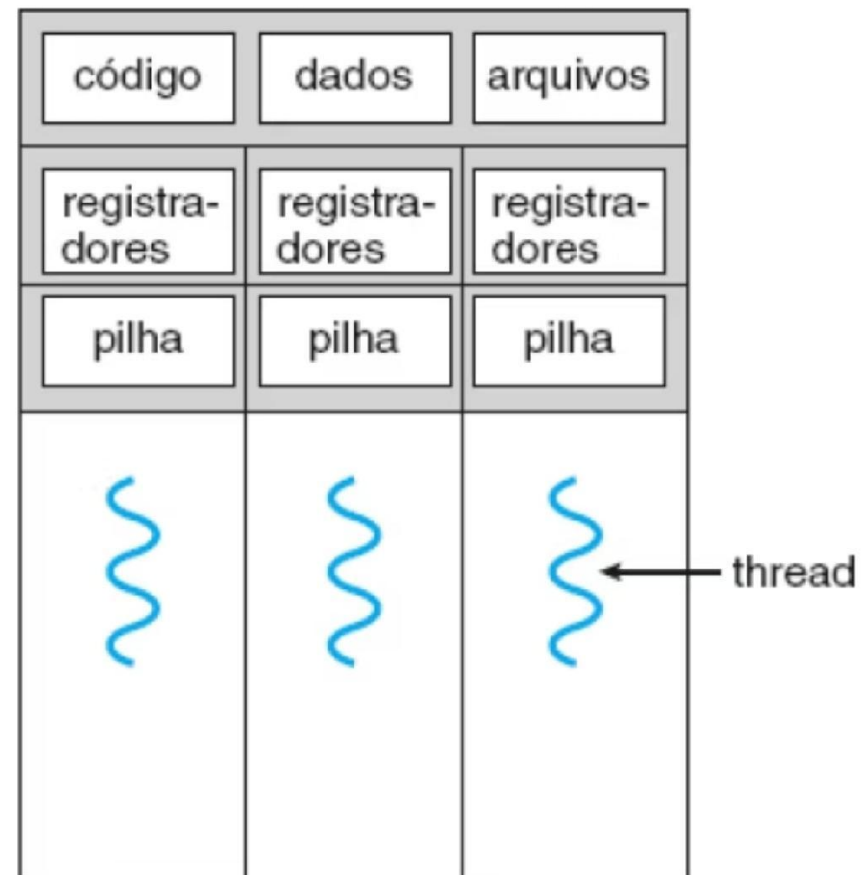
Multithreading

Multithreading é a existência de **múltiplas threads** dentro de um mesmo processo.





processo com um único thread



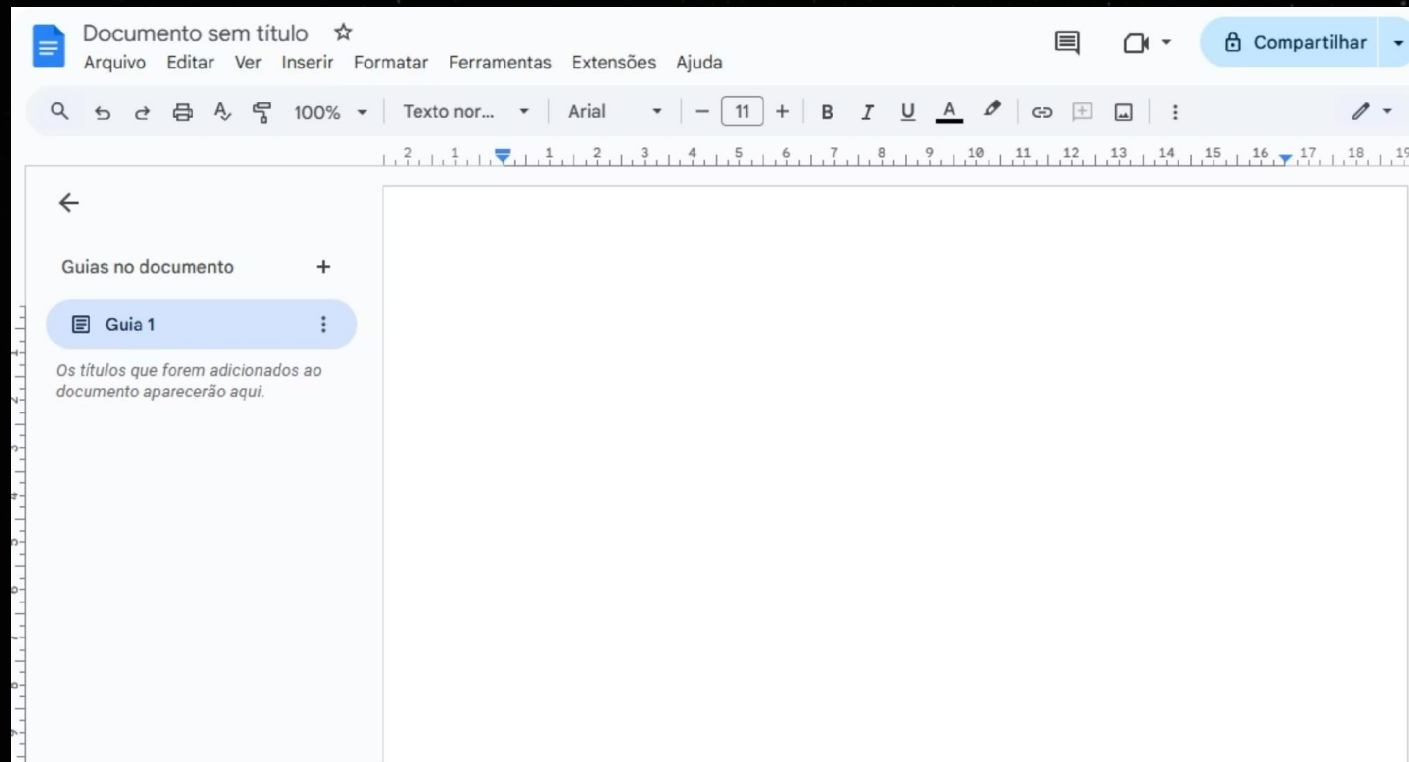
processo com múltiplos threads

Pense em um editor de texto, o que podemos pensar sobre ter **N processos** executando, ao invés de ter **N threads** executando?

Por que usar Threads?

Responsividade

A aplicação pode continuar respondendo mesmo enquanto executa uma tarefa demorada.



Compartilhamento

- Comunicação mais rápida
- Menos necessidade de IPC
- Dados comuns ficam acessíveis

Mas atenção: Dados compartilhados exigem **sincronização**

Economia

Criar e alternar threads costuma ser **mais barato** do que criar e alternar processos.

Mesmo espaço de endereçamento

Menos recursos necessários

Comunicação mais simples

Escalabilidade

Em sistemas com **múltiplos núcleos**, threads podem executar em **paralelo real**.



Processamento de imagens

Cada região da imagem processada por uma thread.



Servidores web

Múltiplas requisições atendidas simultaneamente.



Navegadores

Abas e recursos carregados em paralelo.

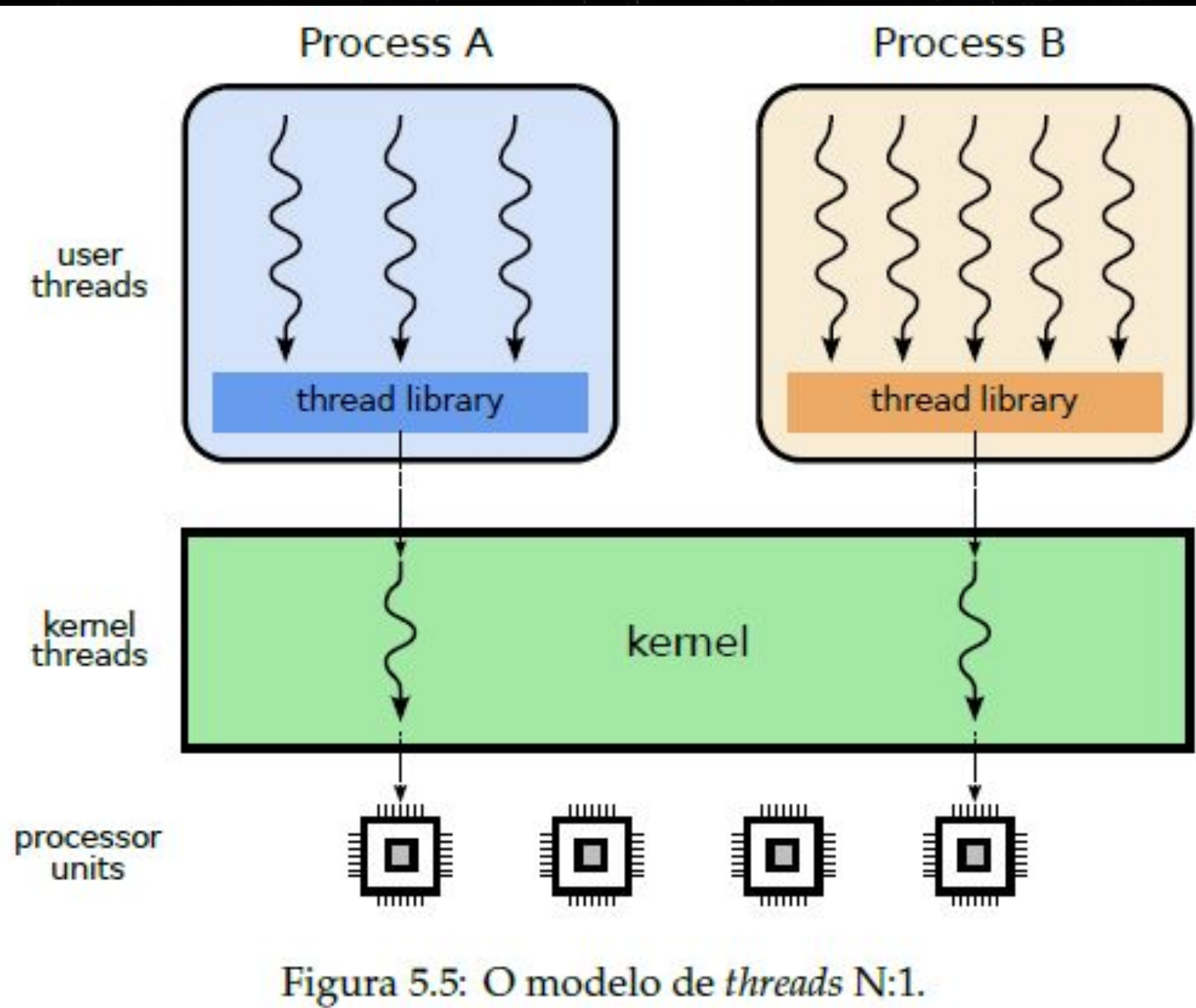
Processos x Threads

Característica	Com processos	Com <i>threads</i> (1:1)	Híbrido
Custo de criação de tarefas	alto	baixo	médio
Troca de contexto	lenta	rápida	variável
Uso de memória	alto	baixo	médio
Compartilhamento de dados entre tarefas	canais de comunicação e áreas de memória compartilhada.	variáveis globais e dinâmicas.	ambos.
Robustez	alta, um erro fica contido no processo.	baixa, um erro pode afetar todas as <i>threads</i> .	média, um erro pode afetar as <i>threads</i> no mesmo processo.
Segurança	alta, cada processo pode executar com usuários e permissões distintas.	baixa, todas as <i>threads</i> herdam as permissões do processo onde executam.	alta, <i>threads</i> que necessitam as mesmas permissões podem ser agrupadas em um mesmo processo.
Exemplos	Servidor Apache (versões 1.*), SGBD PostgreSQL	Servidor Apache (versões 2.*), SGBD MySQL	Navegadores Chrome e Firefox, SGBD Oracle

Tabela 5.3: Comparativo entre uso de processos e de *threads*.

Modelos de Threads

N:1



N:1 - Vantagens

- **Simples gerência:** Leve e de fácil implementação. Como o núcleo somente vê uma thread, a carga de gerência imposta ao núcleo é pequena e não depende do número de threads dentro da aplicação.
- **Exemplos de implementação:** bibliotecas GNU Portable Threads [Engeschall, 2005], User-Mode Scheduling (UMS, Microsoft) e Green Threads (Java)

N:1 - Desvantagens

- **Bloqueante:** se uma thread de usuário solicitar uma leitura do teclado, por exemplo, a thread de núcleo correspondente será suspensa até a conclusão da operação, bloqueando todas as threads daquele processo.
- **Paralelismo:** threads do mesmo processo não podem executar em paralelo;
- **Distribuição de processamento:** núcleo do sistema divide o tempo de processamento entre as threads de núcleo.;

1:1

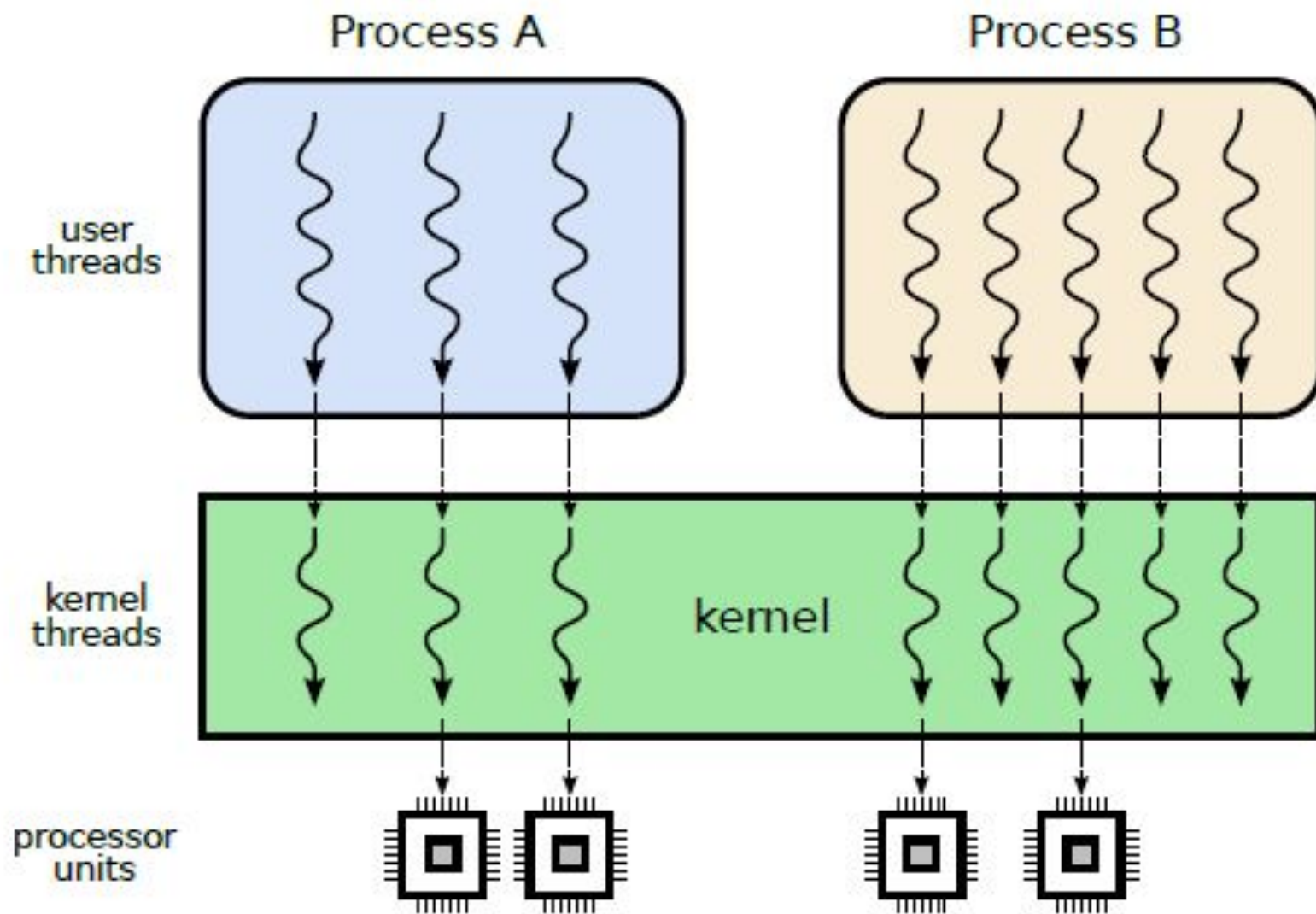


Figura 5.6: O modelo de *threads* 1:1.

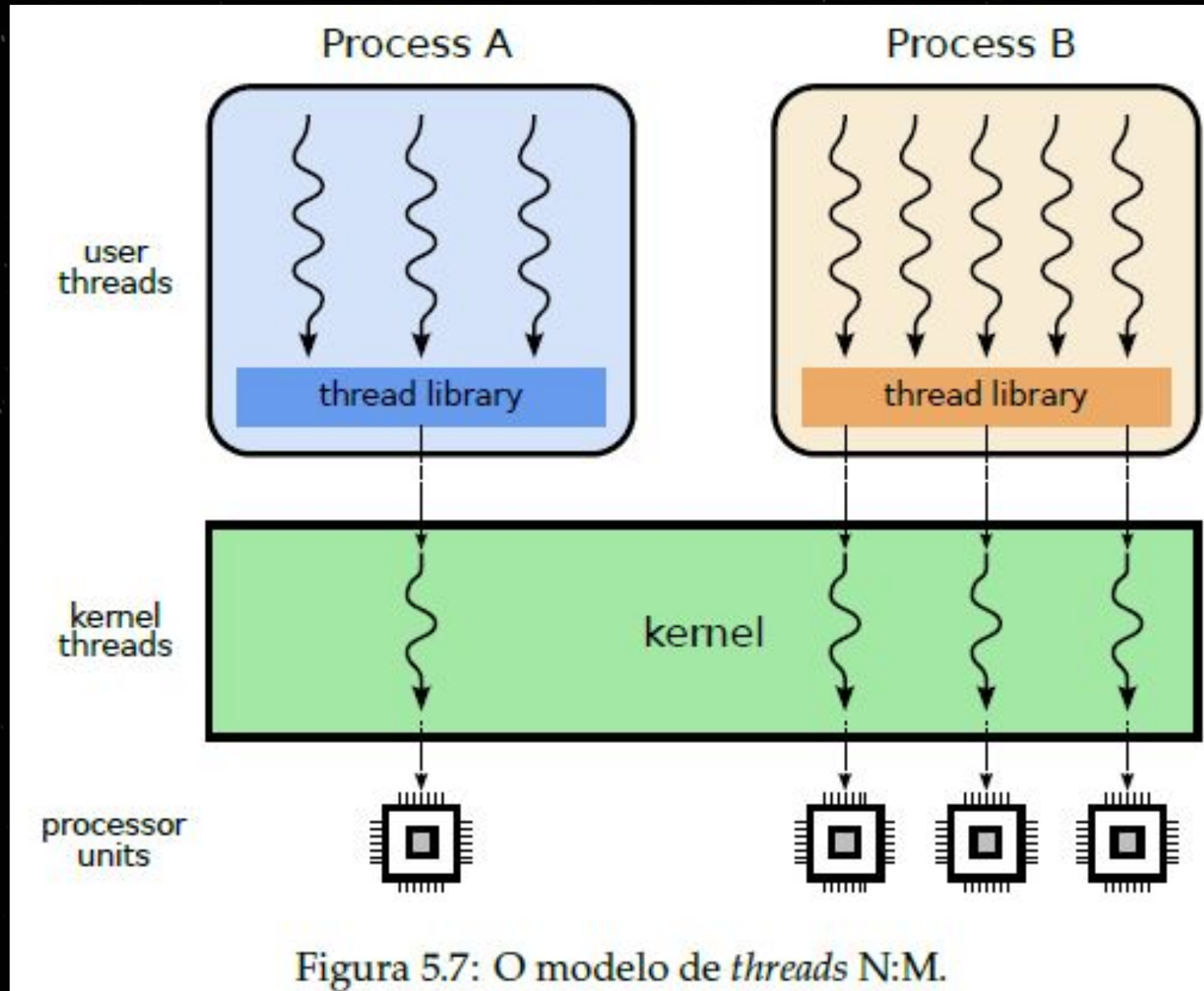
1:1 - Vantagens

- **Não bloqueante:** caso uma thread de usuário solicite uma operação bloqueante, somente sua thread de núcleo correspondente será suspensa, sem afetar as demais threads do processo.
- **Distribuição de processamento:** mais threads do mesmo processo podem executar ao mesmo tempo, o que não era possível no modelo N:1;
- **Exemplos:** modelo mais frequente nos sistemas operacionais atuais, como o Windows NT e seus descendentes, além da maioria dos UNIXes.

1:1 - Desvantagens

- **Pouco escalável:** a criação de um número muito grande de threads impõe uma carga elevada ao núcleo do sistema, inviabilizando aplicações com muitas tarefas (como grandes servidores Web e simulações de grande porte)

N:M



N:M - Vantagens

- **Distribuição de processamento:** alia as vantagens de maior interatividade do modelo 1:1 à maior escalabilidade do modelo N:1;
- **Exemplos:** O modelo N:M é implementado pelo Solaris e também pelo projeto KSE (Kernel-Scheduled Entities) do FreeBSD [Evans and Elischer, 2003].

N:M - Desvantagens

- **Complexidade de implementação;**
- **Custo de Gerência:** maior custo de gerência dos threads de núcleo, quando comparados ao modelo 1:1.

Modelo	N:1	1:1	N:M
Resumo	N <i>threads</i> do processo mapeados em uma <i>thread</i> de núcleo	Cada <i>thread</i> do processo mapeado em uma <i>thread</i> de núcleo	N <i>threads</i> do processo mapeados em M < N <i>threads</i> de núcleo
Implementação	no processo (biblioteca)	no núcleo	em ambos
Complexidade	baixa	média	alta
Custo de gerência	baixo	médio	alto
Escalabilidade	alta	baixa	alta
Paralelismo entre <i>threads</i> do mesmo processo	não	sim	sim
Troca de contexto entre <i>threads</i> do mesmo processo	rápida	lenta	rápida
Divisão de recursos entre tarefas	injusta	justa	variável, pois o mapeamento <i>thread</i> → processador é dinâmico
Exemplos	GNU Portable Threads, Microsoft UMS	Windows, Linux	Solaris, FreeBSD KSE

Tabela 5.2: Comparativo dos modelos de *threads*.

Programando com Threads

No passado, cada sistema operacional definia sua própria interface para a criação de threads, o que levou a problemas de portabilidade das aplicações. Em 1995 foi definido o padrão IEEE POSIX 1003.1c, mais conhecido como **POSIX Threads** ou simplesmente **PThreads** [Nichols et al., 1996], que define uma interface padronizada para o uso de threads na linguagem C.

Biblioteca de Threads: fornece ao programador uma API para criação e gerenciamento de threads.

Exemplos de bibliotecas de threads: a Pthreads do POSIX, Windows Win32, Threads da JVM.

Pthreads POSIX

Pthreads é o padrão POSIX para programação com threads em sistemas UNIX-like.

pthread_create

Cria uma nova thread.

pthread_exit

Encerra a thread atual.

pthread_join

Aguarda o término de outra thread.

pthread_yield

Cede voluntariamente a CPU.

Objetivo: **portabilidade** e **padronização** entre sistemas compatíveis com POSIX.

```
1 // Exemplo de uso de threads Posix em C no Linux
2 // Compilar com gcc exemplo.c -o exemplo -lpthread
3
4 #include <pthread.h>
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <unistd.h>
8
9 #define NUM_THREADS 5
10
11 // cada thread vai executar esta função
12 void *print_hello (void *threadid)
13 {
14     printf ("%ld: Hello World!\n", (long) threadid);
15     sleep (5) ;
16     printf ("%ld: Bye bye World!\n", (long) threadid);
17     pthread_exit (NULL);    // encerra esta thread
18 }
```

```
19 // thread "main" (vai criar as demais threads)
20 int main (int argc, char *argv[])
21 {
22     pthread_t thread[NUM_THREADS];
23     long status, i;
24
25     // cria as demais threads
26     for(i = 0; i < NUM_THREADS; i++)
27     {
28         printf ("Creating thread %ld\n", i);
29         status = pthread_create (&thread[i], NULL, print_hello, (void *) i);
30
31         if (status) // ocorreu um erro
32         {
33             perror ("pthread_create");
34             exit (-1);
35         }
36     }
37
38     // encerra a thread "main"
39     pthread_exit (NULL);
40 }
```

Qual Thread apresentará “Hello World” primeiro?

FIGURA 2.14 Algumas das chamadas de função do Pthreads.

Chamada de thread	Descrição
Pthread_create	Cria um novo thread
Pthread_exit	Conclui a chamada de thread
Pthread_join	Espera que um thread específico seja abandonado
Pthread_yield	Libera a CPU para que outro thread seja executado
Pthread_attr_init	Cria e inicializa uma estrutura de atributos do thread
Pthread_attr_destroy	Remove uma estrutura de atributos do thread

Qual a diferença?

```
for (i = 0; i < NUM_THREADS; i++) {  
    pthread_create(&thread[i], NULL, print_hello, (void *) i);  
    pthread_join(thread[i], NULL);  
}
```

```
for (i = 0; i < NUM_THREADS; i++)  
    pthread_create(&thread[i], NULL, print_hello, (void *) i);  
  
for (i = 0; i < NUM_THREADS; i++)  
    pthread_join(thread[i], NULL);
```

Riscos e Boas Práticas

O risco das threads

Threads compartilham dados. Isso pode gerar **resultados imprevisíveis**.

⚠️ **Condição de corrida**

Duas threads alteram a mesma variável global ao mesmo tempo.

❌ **Inconsistência**

Resultado final depende da ordem de execução não controlada.

🔒 **Sincronização necessária**

Mecanismos como mutex e semáforos resolvem o problema.



Exercícios

- 1 - Forneça dois exemplos de programação em que a criação de múltiplos threads não proporciona melhor desempenho do que uma solução com um único thread.
- 2 - Uma solução com múltiplos threads usando múltiplos threads de nível de usuário pode obter melhor desempenho em um sistema multiprocessador do que em um sistema uniprocessador? Explique.

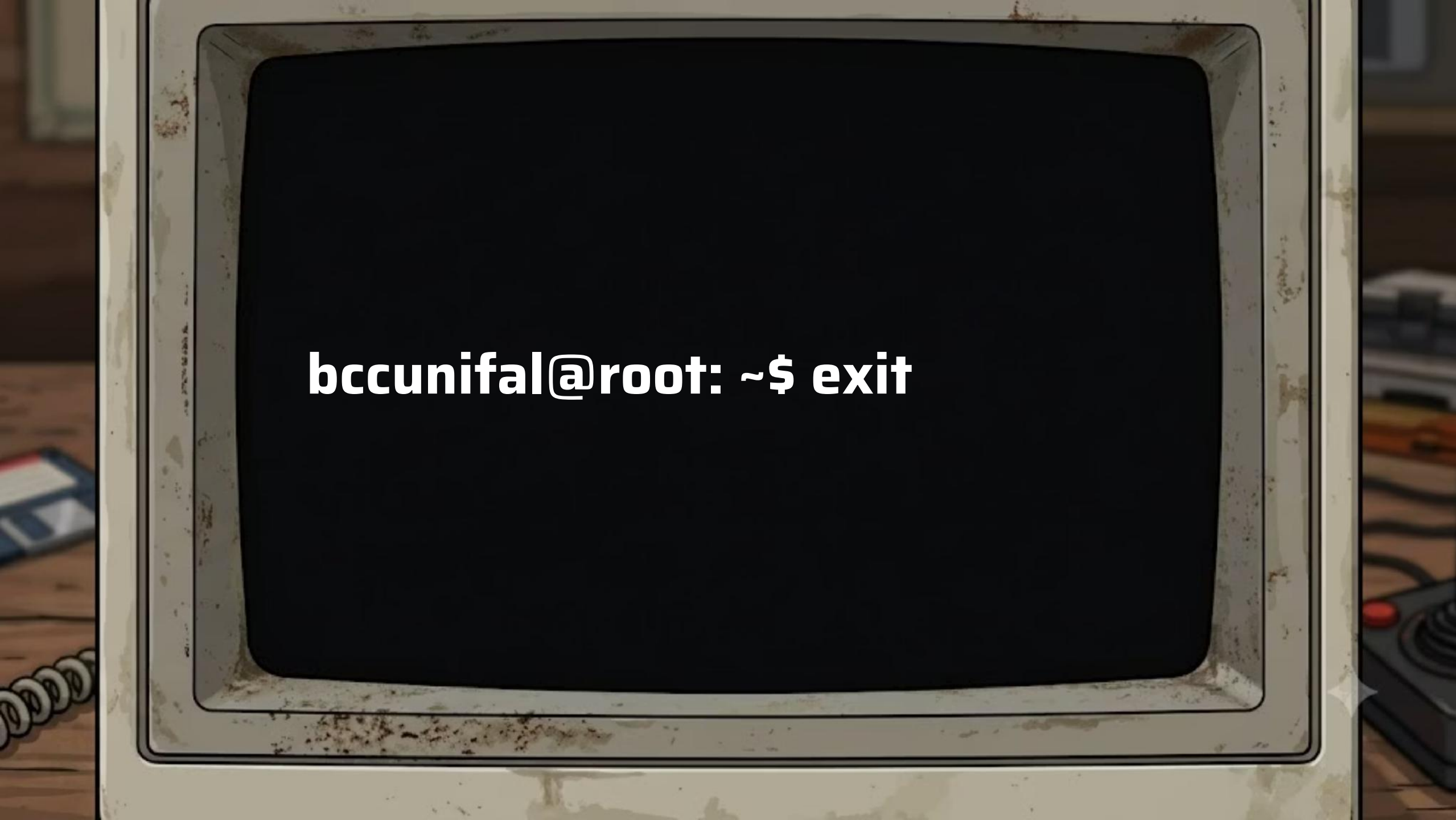
Exercícios

3 - Anteriormente, discutimos o navegador Chrome da Google e sua prática de abrir cada novo website em um processo separado. Os mesmos benefícios seriam alcançados se, em vez disso, a decisão de projeto do Chrome tivesse sido abrir cada novo website em um thread separado? Explique sua resposta.

```
pid_t pid;  
pid = fork();  
if (pid == 0) { /*processo-filho */  
    fork();  
    thread_create( ... );  
}  
fork();
```

A) Quantos processos únicos são criados?

B) Quantos threads únicos são criados?

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the entire image shows a desk with a keyboard and some papers, though they are out of focus.

bccunifal@root: ~\$ exit